Caso práctico - NovaBank

Plan formativo

Rubén Manuel Rodríguez Chamorro

Empresa: NttData

Estudiante de Universidad de Málaga

Ingeniería de la Salud

15 de abril de 2026

Índice general

1. Contexto del Módulo **3**
1.1. Objetivos del Módulo 3

2. Arquitectura por Capas **4**
2.1. Responsabilidades de cada capa: 5
2.2. La regla de dependencias 5
2.3. Estructura del código 6

3. De la memoria a la base de datos **8**
3.1. El contrato del repositorio 8
3.1.1. Mapeo de ResultSet a objetos Java 8
3.1.2. Se modifica o permanece 8

4. Gestión de Conexiones a la Base de Datos **10**
4.1. Gestión de conexiones a la base de datos 10
4.1.1. DatabaseConnectionManager 10

5. Patrón Repository **11**

6. Transacciones JDBC **13**
6.1. El servicio controla la transacción 13
6.2. Variantes Transaccionales 13

7. Patrones de Diseño **14**
7.1. Patrón Singleton 14
7.2. Patrón Factory 15
7.3. Patrón Builder 16
7.4. Patrón Strategy 18
7.5. Patrón Decorator 19
7.5.1. Chevrolet Nova 21
7.6. Programación funcional, Streams y Lambdas 21

8. Pruebas del Sistema (Testing) **22**
8.1. Testing en el Módulo 2 22
8.1.1. Tests de servicios con Mockito 22
8.1.2. Tests de integración con base de datos 22

9. Limitaciones y trabajo futuro **24**

10. Conclusiones **25**

Capítulo 1

Contexto del Módulo

En el módulo del Caso Práctico 1 se construyó una primera versión del sistema NovaBank como aplicación de consola en Java. En este módulo, el objetivo real es introducirle a la aplicación persistencia para que los datos se mantengan en el programa una vez éste haya sido desconectado.

Por lo que se llevará a cabo una sustitución de los Map, List y HashMap anteriormente utilizados, para conectarle una base de datos real, en nuestro caso usaremos **PostgreSQL**

1.1. Objetivos del Módulo

Como objetivos finales de este módulo se nos propone:

- Refactorizar el sistema adoptando arquitectura por capas
- Aplicar patrón **Repository** para aislar el acceso a datos.
- Introducir y comprender patrones de diseño: **Singleton, Factory, Strategy y Builder**
- Aplicar programación funcional, **uso de Streams y Lambdas** de Java.
- Gestión de transacciones JDBC.
- Mantener y ampliar los test del módulo anterior.

Capítulo 2

Arquitectura por Capas

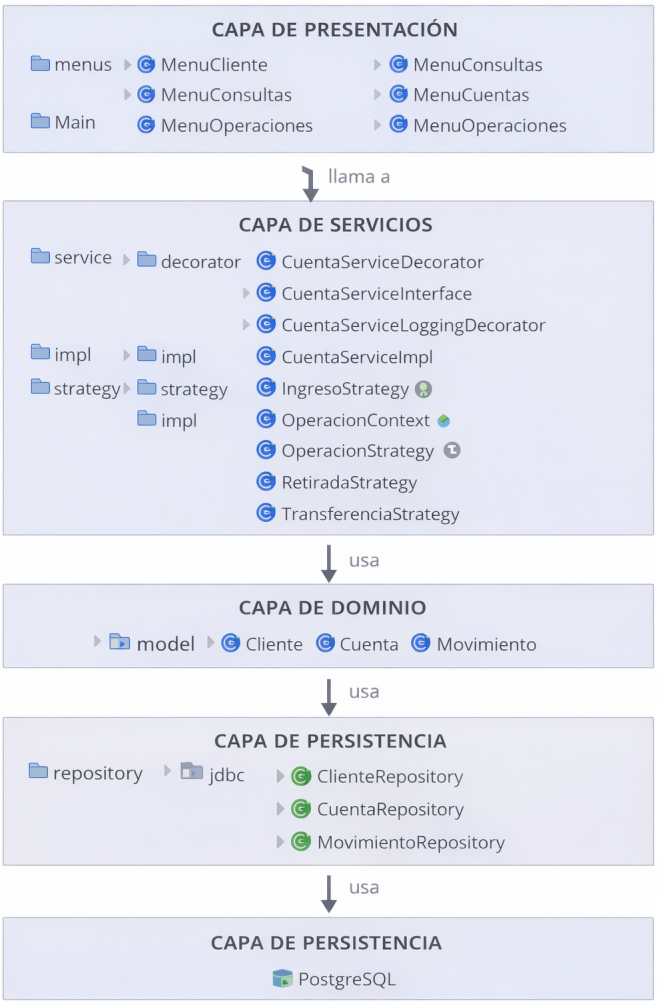

Figura 2.1: Capas del Sistema

2.1. Responsabilidades de cada capa:

- **Capa de presentación:** Es la única capa que interactúa con el usuario. Únicamente muestra los menús que correspondan, obtiene información desde la consola y presenta los resultados.

- **Capa de Servicios:** Aquí viene la lógica de negocio, los servicios no saben nada de cómo se muestran los datos al usuario, simplemente se apoyan en los repositorios para cumplir con la persistencia.

- **Capa de dominio:** Sólo contiene los conceptos del negocio bancario, son clases muy simples, sólo contienen sus atributos, getters y setters, no contienen ninguna lógica de negocio ni acceso a datos. [Docs, 2024]

- **Capa de persistencia:** Se implementa el patrón **repository**, el objetivo es, para cada entidad del dominio existe un repositorio responsable de las operaciones de lectura y escritura. El patrón Repository tiene una consecuencia importante para la testabilidad, los servicios pueden probarse de forma completamente aislada.

2.2. La regla de dependencias

Tras la creación de todas las clases y paquetes necesarios para que este programa cumpla, tanto con los requisitos solicitados, como con las expectativas esperadas de funcionalidad.

- **Menús:** La capa presentación llama a los servicios y usa entidades del dominio, pero en nningún caso accede directamente a la presistencia ni contiene lógica de negocio.

- **Services:** cumple con la arquitectura, todas las clases contienen exclusivamente la lógica de negocio del sistema, incluyendo validaciones, reglas del dominio y la coordinación de operaciones complejas. Tanto el patrón **Strategy** que encapsula las operaciones financieras, modificando sólo el estado de las entidades y el patrón **Decorator** añade funcionalidades transversales como el registro de operaciones sin alterar la lógica principal ni romper la separación entre capas.

- **Repository:** cumple la arquitectura por capas definida en el módulo. Todas las clases del paquete `repository` y sus implementaciones JDBC se encargan exclusivamente del acceso a datos, aplicando el patrón *Repository* de forma correcta. Esta capa contiene únicamente consultas SQL, operaciones JDBC y

el mapeo de *ResultSet* a entidades del dominio, sin incluir lógica de negocio ni referencias a la capa de servicios o de presentación.

2.3. Estructura del código

En el Módulo 1 la aplicación disponía de una única clase de menú que concentraba toda la interacción con el usuario. En el Módulo 2, el Caso Práctico plantea la posibilidad de separar la presentación en dos paquetes diferenciados: `controller` y `view`. Sin embargo, esta separación solo resulta necesaria cuando existen controladores explícitos que coordinan la comunicación entre la vista y los servicios.

En NovaBank no existe una capa de controladores independiente, ya que los menús actúan directamente como vista y delegan todas las operaciones en la capa de servicios sin contener lógica de negocio. Por este motivo, no es necesario crear un paquete `controller` artificialmente. Toda la presentación se agrupa en el paquete `menus`, que cumple el rol de `view` dentro de la arquitectura por capas.

Esta decisión es coherente con el enfoque de organización por capa técnica, donde cada paquete representa una capa de la arquitectura y se evita introducir estructuras adicionales que no aportan funcionalidad real al proyecto.

```
com.novabank
├── config
├── Main.java
├── exception
├── menus
│   ├── MenuCliente.java
│   ├── MenuCuentas.java
│   ├── MenuOperaciones.java
│   └── MenuConsultas.java
├── model
│   ├── Cliente.java
│   ├── Cuenta.java
│   ├── Movimiento.java
│   └── TipoMovimiento.java
├── repository
│   ├── ClienteRepository.java
│   ├── CuentaRepository.java
│   ├── MovimientoRepository.java
│   └── jdbc
│       ├── ClienteRepositoryJdbc.java
│       ├── CuentaRepositoryJdbc.java
│       └── MovimientoRepositoryJdbc.java
└── service
    ├── ClienteService.java
    ├── ConsultaService.java
    ├── CuentaServiceInterface.java
    ├── impl
    │   └── CuentaServiceImpl.java
    ├── decorator
    │   ├── CuentaServiceDecorator.java
    │   └── CuentaServiceLoggingDecorator.java
    └── strategy
        ├── OperacionStrategy.java
        ├── OperacionContext.java
        ├── IngresoStrategy.java
        ├── RetiradaStrategy.java
        └── TransferenciaStrategy.java
```

Capítulo 3

De la memoria a la base de datos

El cambio fundamental de este módulo consiste en sustituir el almacenamiento en memoria del Módulo 1 por una persistencia real mediante PostgreSQL. Sin embargo, gracias a la arquitectura por capas, el impacto de este cambio se limita exclusivamente a la capa de persistencia. El resto del sistema continúa funcionando sin modificaciones, lo que demuestra que el diseño desacoplado es correcto.

Anteriormente, los repositorios utilizaban estructuras `HashMap` para almacenar los datos. En esas mismas clases pasan a ejecutar consultas SQL mediante JDBC.

3.1. El contrato del repositorio

Las interfaces de repositorio no han cambiado entre módulos. En el Módulo 1, un método como `buscarPorId` consultaba un `Map`; en el Módulo 2, ejecuta una sentencia SQL, pero la firma del método es exactamente la misma. Esto garantiza que la capa de servicios no necesita ser modificada durante la migración.

3.1.1. Mapeo de ResultSet a objetos Java

Con JDBC, los resultados de una consulta llegan en forma de `ResultSet`. Cada repositorio es responsable de convertir cada fila en la entidad correspondiente del dominio. Este proceso de mapeo es manual en JDBC, asignando explícitamente cada columna a un atributo del objeto. En módulos posteriores, como con Spring Data JPA, este proceso será automatizado mediante anotaciones.

3.1.2. Se modifica o permanece

La migración afecta únicamente a la capa de persistencia. La siguiente tabla resume los elementos que se modifican y los que permanecen igual:

Se modifica	No se modifica
Implementaciones de repositorio (de `Map` a JDBC)	Interfaces de repositorio (el contrato con los servicios)
Dependencias Maven: se añade el driver de PostgreSQL	Servicios: siguen llamando a los mismos métodos
Clase gestora de conexiones	Entidades del dominio (`Cliente`, `Cuenta`, `Movimiento`)
Creación de la base de datos y ejecución de `schema.sql`	Capa de presentación y menús
Algunos métodos pasan a devolver `Optional<T>`	Tests de servicios con Mockito

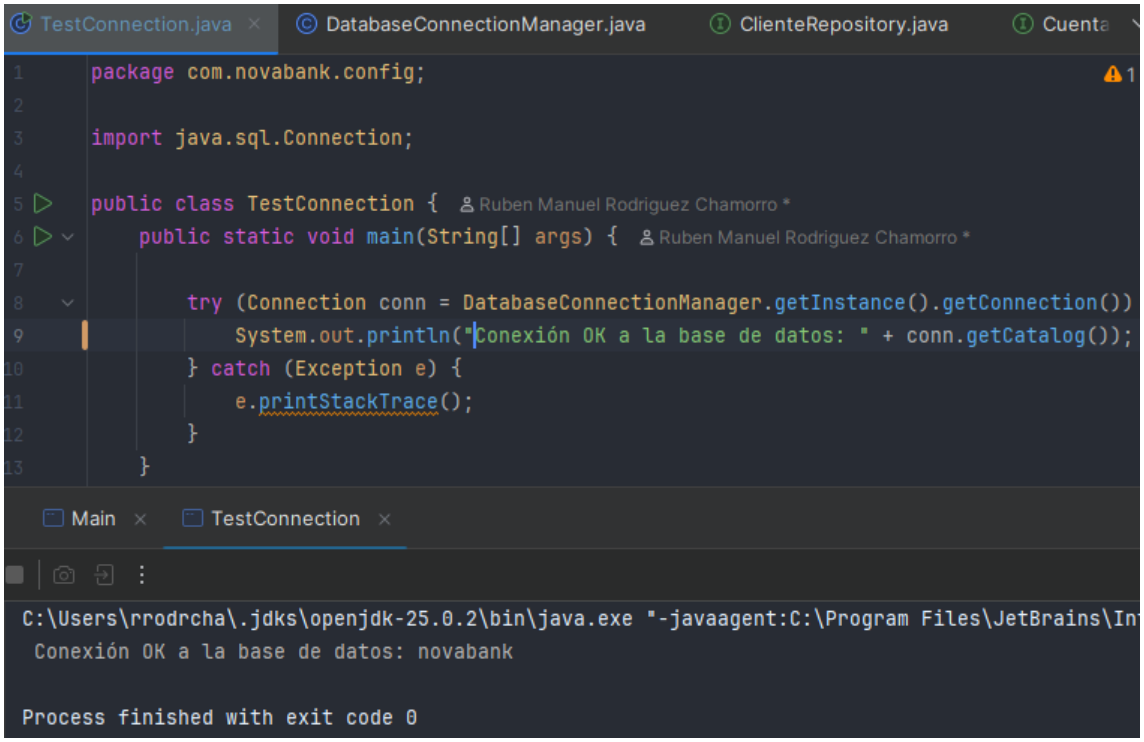

Figura 3.1: Comprobación de conexión

Capítulo 4

Gestión de Conexiones a la Base de Datos

4.1. Gestión de conexiones a la base de datos

> Para evitar duplicar código y garantizar una configuración centralizada. Todas las operaciones JDBC requieren una conexión activa a PostgreSQL.

La creación de conexiones se concentra en una única clase `DatabaseConnectionManager`. Esta clase actúa como punto único de acceso a la base de datos y aplica el patrón *Singleton*, tal como exige el Caso Práctico 2.

4.1.1. DatabaseConnectionManager

El patrón *Singleton* se aplica al gestor de conexiones, no a las conexiones individuales. Esto significa que existe una única instancia del gestor durante toda la ejecución del programa, pero cada llamada a `getConnection()` crea una nueva conexión física a PostgreSQL. Este comportamiento es el adecuado, ya que las conexiones JDBC no deben compartirse entre operaciones concurrentes.

El gestor centraliza la URL, el usuario y la contraseña, evitando que cada repositorio tenga que conocer estos detalles. Además, el constructor es privado, lo que impide crear instancias adicionales fuera de la clase y garantiza que toda la aplicación utilice el mismo punto de acceso.

> Todas las conexiones deben cerrarse correctamente tras su uso, evitando fugas de recursos, manteniendo el sistema estable. Para ello, los repositorios emplean la estructura *try-with-resources*, que garantiza el cierre automático de la conexión, el `PreparedStatement` y el `ResultSet`, incluso si ocurre una excepción durante la ejecución.

Capítulo 5

Patrón Repository

El patrón **Repository** se utiliza en NovaBank para separar la lógica de negocio del acceso a datos. En lugar de que los servicios trabajen directamente con SQL o con `Connection`, cada entidad del dominio (Cliente, Cuenta y Movimiento) dispone de un repositorio que actúa como intermediario entre la aplicación y PostgreSQL.

Esta separación permite mantener una arquitectura limpia: los servicios solo expresan intenciones de negocio (crear un cliente, actualizar un saldo, registrar un movimiento), mientras que los repositorios se encargan de ejecutar las consultas SQL necesarias.

Estructura aplicada en NovaBank

El proyecto define una interfaz por cada agregado del dominio:

- **ClienteRepository**
- **CuentaRepository**
- **MovimientoRepository**

Estas interfaces representan las operaciones que la aplicación necesita realizar sobre cada entidad, sin importar cómo se implementen. De este modo, la lógica de negocio no depende de PostgreSQL ni de JDBC.

Las implementaciones concretas se encuentran en el paquete `repository.jdbc`, donde cada clase contiene las consultas SQL y la gestión de conexiones. Esta organización permite ver claramente la separación entre:

- **Interfaces**: definen el contrato.
- **Implementaciones JDBC**: ejecutan el SQL real contra la base de datos.

Métodos transaccionales

En los repositorios de cuentas y movimientos se incluyen variantes de algunos métodos que aceptan una `Connection` activa. Estas versiones se utilizan cuando una operación requiere formar parte de una transacción mayor, como ocurre en las transferencias:

- lectura de la cuenta origen,
- actualización de su saldo,
- lectura de la cuenta destino,
- actualización de su saldo,
- registro de ambos movimientos.

Todas estas acciones deben ejecutarse dentro de la misma transacción para garantizar la consistencia. Por ello, el servicio abre la conexión, desactiva el auto-commit y pasa esa misma conexión a los repositorios. Las variantes sin conexión se utilizan para operaciones simples que no requieren transacción.

Beneficios obtenidos

La aplicación del patrón Repository en NovaBank aporta:

- **Aislamiento del acceso a datos**: los servicios no contienen SQL.
- **Código más limpio y mantenible**: cada capa tiene una responsabilidad clara.
- **Facilidad para cambiar la fuente de datos**: solo habría que sustituir las implementaciones JDBC.
- **Soporte para transacciones complejas**: gracias a los métodos que aceptan una `Connection`.

En conjunto, esta estructura permite que la lógica bancaria se mantenga independiente del mecanismo de persistencia y que el sistema sea más robusto y extensible.

Capítulo 6

Transacciones JDBC

En un sistema bancario, las transferencias entre las cuentas, representan el ejemplo más claro de aquellas operaciones que se deben ejecutar de manera conjunta y completarse todas o no se completa ninguna.

6.1. El servicio controla la transacción

En la clase *CuentaServiceImpl*, la operación *trasnferir():*

- Abre una conexión con DatabaseConnectionManager.
- Desactiva el autocommit.
- Ejecuta cada paso que hay que realizar en la trasnferencia haciendo uso de la misma conexión.
- Hace *commit()* si todo va bien o *rollback()* si algo falla.

6.2. Variantes Transaccionales

Se ha necesitado usar *CuentaRepositoryJDBC y MovimientoRepositoryJDBC* tienen métodos como:

- *buscarPorNumero(String numero, Connection conn)*
- *actualizarSaldo(Long id, double saldo, Connection conn)*

Estas variantes existen y son necesarias porque el servicio necesita que todas las operaciones usen la misma conexión. Además, nos permite que los repositorios accedan a PostgreSQL, que se abran las conexiones, ejecuten consultas SQL, transformar los resultados en objetos del dominio y garantizar transacciones atómicas.

Capítulo 7

Patrones de Diseño

> Los patrones de diseño son soluciones probadas a problemas recurrentes cuando desarrollamos Software. Su utilidad son reducir acoplamiento o garantizar un comportamiento correcto. [Gamma et al., 1994]

7.1. Patrón Singleton

Este patrón se ha aplicado en la clase **DatabaseConnectionManager**, en la implementación este patrón garantiza que exista un único gestor centralizado de configuración de la base de datos durante toda la ejecución del programa. Esto evita que cada repositorio tenga que conocer la URL, el usuario o la contraseña, y asegura que todos utilicen el mismo punto de acceso. [Guru, 2024c]

- **Constructor privado**, esto impide crear instancias desde fuera.

```
private DatabaseConnectionManager() {}
```

- Mantiene una instancia estática única.

```
private static DatabaseConnectionManager instance;
```

- Proporciona un **método público y sincronizado** *getInstance()*, que siempre devuelve la misma instancia.

```

public static synchronized DatabaseConnectionManager
    getInstance() {
     if (instance == null) {
         instance = new DatabaseConnectionManager();
     }return instance;}
```

- No convierte la conexión en **"Singleton"**, sino solo en un gestor, mientras que cada llamada a *getConnection()* devuelve una conexión nueva y efímera, como exige JDBC.

```
public Connection getConnection() {
    return DriverManager.getConnection(URL, USER, PASSWORD);}
```

7.2. Patrón Factory

El patrón Factory no modifica la lógica del programa ni introduce un comportamiento nuevo; simplemente concentra en un único lugar la creación de movimientos según su tipo. Esto hace que el código sea más claro y que la intención de cada operación quede mejor expresada. [Guru, 2024b]

En NovaBank se ha aplicado el patrón **Factory** para organizar la creación de objetos `Movimiento`. Antes de introducir este patrón, los movimientos se creaban directamente en el servicio mediante llamadas al constructor, repitiendo la misma instrucción en varios métodos. Aunque esto funcionaba correctamente, se decidió extraer esa creación a una clase dedicada para mantener el código más ordenado y evitar repetir la misma línea en distintos puntos del servicio.

Implementación

La clase `MovimientoFactory` contiene métodos estáticos que devuelven un movimiento ya configurado según el tipo de operación realizada.

```
public static Movimiento crearIngreso(Long cuentaId, double
    cantidad) {
    return new Movimiento(cuentaId, TipoMovimiento.DEPOSITO,
    cantidad);
}
```

```
public static Movimiento crearTransferenciaSaliente(Long
    cuentaId, double cantidad) {
    return new Movimiento(cuentaId, TipoMovimiento.
    TRANSFERENCIA_SALIENTE, cantidad);
}
```

Uso en el servicio

Antes de aplicar el patrón, el servicio creaba los movimientos directamente:

```
// Antes
new Movimiento(cuenta.getId(), TipoMovimiento.DEPOSITO,
    cantidad);
```

Tras introducir la Factory, el servicio delega esa creación:

```
// Despues
MovimientoFactory.crearIngreso(cuenta.getId(), cantidad);
```

El comportamiento es exactamente el mismo, pero la creación queda centralizada y el código del servicio resulta más limpio y fácil de leer.

7.3. Patrón Builder

El patrón **Builder** se ha aplicado únicamente en la clase `Cliente`. Antes, esta clase tenía varios constructores con parámetros, pero al simplificarla y dejar solo el constructor vacío (requerido por JDBC), era necesario disponer de una forma clara de crear objetos con todos sus atributos. El Builder permite construir un cliente de manera ordenada mediante métodos encadenados, sin necesidad de múltiples constructores.

Implementación

La clase incluye una clase interna `Builder` con métodos para establecer cada campo y un método final `build()` que devuelve el objeto configurado:

```
public static class Builder {
    private Long id;
    private String nombre;
    private String apellidos;
    private String dni;
    private String email;
    private String telefono;
```

```java
    public Builder nombre(String n) { this.nombre = n; return this; }
    public Builder apellidos(String a) { this.apellidos = a; return this; }
    public Builder dni(String d) { this.dni = d; return this; }
    public Builder email(String e) { this.email = e; return this; }
    public Builder telefono(String t) { this.telefono = t; return this; }

    public Cliente build() {
        Cliente c = new Cliente();
        c.setId(id);
        c.setNombre(nombre);
        c.setApellidos(apellidos);
        c.setDni(dni);
        c.setEmail(email);
        c.setTelefono(telefono);
        return c;
    }
}
```

Uso en el servicio

```java
Cliente cliente = new Cliente.Builder()
        .nombre(nombre)
        .apellidos(apellidos)
        .dni(dni)
        .email(email)
        .telefono(telefono)
        .build();
```

Uso en el repositorio

```java
return new Cliente.Builder()
        .id(rs.getLong("id"))
        .nombre(rs.getString("nombre"))
        .apellidos(rs.getString("apellidos"))
        .dni(rs.getString("dni"))
        .email(rs.getString("email"))
        .telefono(rs.getString("telefono"))
        .build();
```

En resumen, el Builder se utiliza para mantener la clase `Cliente` simple y, al mismo tiempo, permitir una creación clara y consistente de objetos tanto desde el servicio como desde la base de datos. [Guru, 2024a]

El resto de clases de nuestro modelo `Cuenta` y `Movimientos`, no implementan **Builder** porque su creación es simple, determinista y no presenta combinaciones complejas de parámetros. En el caso de `Movimiento`, la creación está centralizada mediante el patrón **Factory**, que es la solución adecuada para este tipo de entidad. Aunque sería técnicamente posible añadir un **Builder** a Cuenta, no aportaría beneficios reales y solo añadiría complejidad innecesaria.

7.4. Patrón Strategy

El patrón **Strategy** se ha utilizado para separar las distintas formas de modificar el saldo de una cuenta según el tipo de operación realizada. En lugar de concentrar toda la lógica en el servicio mediante condicionales, cada comportamiento se encapsula en una estrategia independiente. Esto hace que el código sea más claro y que cada operación tenga su propia clase. [Guru, 2024d]

Implementación

Se define una interfaz común para todas las operaciones:

```
public interface OperacionStrategy {
    void ejecutar(Cuenta cuenta, double cantidad);}
```

Cada operación implementa su propia estrategia:

```
public class IngresoStrategy implements OperacionStrategy {
    public void ejecutar(Cuenta cuenta, double cantidad) {
        cuenta.setSaldo(cuenta.getSaldo() + cantidad);
    }}
```

```
public class RetiradaStrategy implements OperacionStrategy {
    public void ejecutar(Cuenta cuenta, double cantidad) {
        if (cuenta.getSaldo() < cantidad)
            throw new IllegalArgumentException("Saldo
   insuficiente.");
        cuenta.setSaldo(cuenta.getSaldo() - cantidad);
    }
}
```

El contexto permite seleccionar la estrategia adecuada en cada momento:

```java
public class OperacionContext {
    private OperacionStrategy strategy;
    public void setStrategy(OperacionStrategy s) { this.
   strategy = s; }
    public void ejecutar(Cuenta c, double cantidad) { strategy.
   ejecutar(c, cantidad); }
}
```

Uso en el servicio

El servicio elige la estrategia según la operación:

```java
OperacionContext ctx = new OperacionContext();
ctx.setStrategy(new IngresoStrategy());
ctx.ejecutar(cuenta, cantidad);
```

En resumen, Strategy permite que cada operación bancaria tenga su propia lógica aislada, manteniendo el servicio más limpio y evitando condicionales repetidos.

7.5. Patrón Decorator

En NovaBank se aplica el patrón **Decorator** para añadir una capa visual a las operaciones del servicio de cuentas sin modificar la lógica de negocio. El objetivo es separar claramente dos responsabilidades: (1) la gestión bancaria real, implementada en `CuentaServiceImpl`, y (2) la presentación estética en consola, gestionada por el decorador.

Estructura del Decorator

Dado que en este proyecto, la creación de un patrón **Decorator**, tenía pocas funcionalidades, se ha decidido crear una pequeña implementación cuando se lleven a cabo operaciones en las cuentas de los clientes, es decir, el cliente cuando realice un ingreso, un retiro o una trasnferencia, verá una animación de que el proceso se está llevando a cabo.

Dado que nuestro banco se llama NovaBank como los **Chevrolet Nova**, he decidido crear un efecto visual cuando alguno de estos procesos se esté llevando a cabo.

Decorador visual

El decorador concreto `CuentaServiceVisualDecorator` añade únicamente efectos visuales que, en este caso sólo pueden sustituir pequeñas animaciones de consola pero que tienen un gran potencial de desarrollo para aplicar a futuras implementaciones.

```
private void animarCocheProgreso(int ancho, int pasos, int
    delay) {
    for (int i = 0; i <= pasos; i++) {
        double progreso = (double) i / pasos;
        int pos = (int) (progreso * ancho);

        StringBuilder barra = new StringBuilder("|");
        for (int j = 0; j < ancho; j++)
            barra.append(j == pos ? "" : "-");

        barra.append("| ").append((int)(progreso * 100)).append
    ("%");
        System.out.print("\r" + barra);
        Thread.sleep(delay);
    }
}
```

Uso en la aplicación

El decorador se aplica envolviendo la implementación real del servicio:

```
CuentaServiceInterface servicio =
    new CuentaServiceVisualDecorator(
        new CuentaServiceImpl(cuentaRepository, clienteService,
    movimientoRepository)
    );
```

De este modo, la lógica de negocio permanece limpia y el comportamiento visual puede activarse o retirarse sin modificar el servicio principal.

7.5.1. Chevrolet Nova

> El Chevy argentino, derivado del Nova, es considerado un coche clásico y coleccionable, aún admirado por entusiastas del automovilismo, este coche creado en 1911 fue un referente en automovilísmo de la época. Dado que nuestro banco se llama **Nova**, se ha realizado un pequeño homenage en la consola añadiendo este famoso clásico.

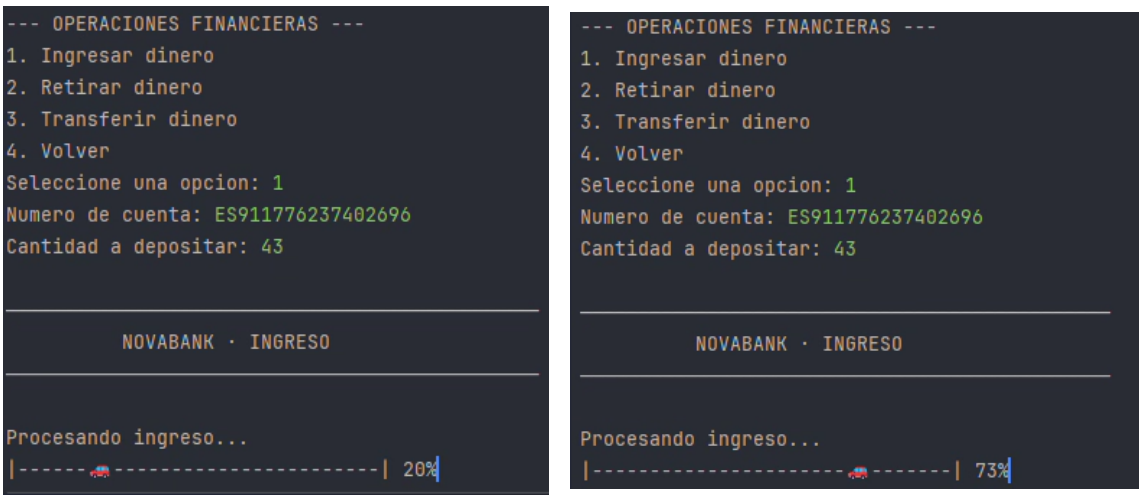

Figura 7.1: Coche Chevrolet Nova cargando la barra de progreso.

7.6. Programación funcional, Streams y Lambdas

En NovaBank se utilizan **Streams** y **expresiones lambda** para procesar colecciones de forma más clara y expresiva. Aunque la mayor parte del filtrado y ordenación se realiza directamente en PostgreSQL mediante consultas JDBC, los Streams se emplean en aquellas operaciones que sí trabajan con listas ya cargadas en memoria, como la ordenación de cuentas, la transformación de datos o el filtrado de movimientos ya obtenidos.

Su uso se limita a los casos donde mejoran la legibilidad, siguiendo la recomendación del módulo de no emplearlos de forma innecesaria.

Capítulo 8

Pruebas del Sistema (Testing)

8.1. Testing en el Módulo 2

La suite de tests del Módulo 1 se ha eliminado, puesto que los test se validaban en memoria local y la lógica de negocio ha cambiado en su totalidad. En particular, los servicios se han probado de forma aislada mediante Mockito, mientras que los repositorios JDBC se han validado mediante pruebas de integración contra una base de datos PostgreSQL real.

8.1.1. Tests de servicios con Mockito

Los servicios se han probado sin depender de la base de datos, simulando los repositorios mediante Mockito. Esto permite verificar exclusivamente la lógica de negocio, asegurando que las reglas implementadas en los servicios funcionan correctamente de forma aislada.

Este enfoque se ha aplicado también a los métodos de creación de cuentas, ingreso, retirada y validaciones de búsqueda. Los métodos de transferencia no se han probado con Mockito debido a que utilizan conexiones JDBC reales y requieren transacciones, por lo que se han incluido en las pruebas de integración. [Torres et al., 2017]

8.1.2. Tests de integración con base de datos

Los repositorios JDBC se han probado directamente contra una base de datos PostgreSQL real. Antes de cada prueba se realiza un `TRUNCATE` de las tablas implicadas para garantizar independencia entre tests y evitar efectos colaterales.

El uso del patrón *Builder* ha simplificado la creación de objetos de prueba, mejorando la legibilidad y mantenibilidad de los tests.

Se han implementado pruebas de integración para:

- **ClienteRepositoryJdbc**: inserción, búsqueda por ID, búsqueda por DNI y listado.
- **CuentaRepositoryJdbc**: inserción, búsqueda por número, listado por cliente, actualización de saldo y variante transaccional. [Thakare and Kiwelekar, 2021]
- **MovimientoRepositoryJdbc**: inserción, listado por cuenta y listado por rango de fechas, utilizando los tipos reales definidos en `MovimientoFactory` (`DEPOSITO`, `RETIRO`, `TRANSFERENCIA_SALIENTE`, `TRANSFERENCIA_ENTRANTE`).
- **TransferenciaIntegrationTest**: prueba no completada.

Capítulo 9

Limitaciones y trabajo futuro

Aunque se ha cubierto la mayor parte del sistema, existen dos limitaciones importantes dentro del sistema.

No se ha conseguido probar todos los test unitarios para comprobar cada una de las validaciones del sistema, se ha realizado un enfoque en este proyecto destinado más a cubrir la mayoría de patrones que se solicitaban, comprenderlos e intentar darle un enfoque en el proyecto. Esto ha ocasionado que los Test, que en principio son lo más indispensable para verificar que el código se ha completado y sigue la lógica de negocio que se ha construido previamente, no hayan sido testeados en su totalidad o haya ciertas validaciones que falten por completar.

Además, los errores en consola cuando se asigna un cliente incorrecto, cuando se introduce un ingreso incorrecto o, resumiendo, se realiza una operación que no es la indicada en ese momento, entonces los errores que aparecen en pantalla no están controlados, en cuanto al mensaje que aparece.

Para futuras entregas o bien modificaciones a realizar de ésta entrega, se tiene: [Zhao et al., 2024]

1. El repaso de los test, para cubrir todas las validaciones.
2. Realizar un control más exhaustivo de los errores que aparecen por pantalla.
3. Hacer uso del patrón Decorator de una manera más profesional, usándolo en cada una de las pantallas de consola y no sólo en **ingreso**, **retirada** y **trasnferencia**

Capítulo 10

Conclusiones

El desarrollo del Módulo 2 ha supuesto una mejora importante en NovaBank, pasando de un sistema en memoria a uno persistente, modular y escalable. La arquitectura por capas ha permitido separar responsabilidades, facilitando el mantenimiento y las pruebas, y ha hecho posible migrar a PostgreSQL modificando solo la capa de persistencia.

El uso de patrones de diseño como Singleton, Factory, Builder, Strategy y Decorator ha mejorado la organización, claridad y extensibilidad del código, evitando redundancias y facilitando la incorporación de nuevas funcionalidades.

En cuanto al testing, se han combinado pruebas unitarias con Mockito para validar la lógica de negocio, y pruebas de integración con PostgreSQL para asegurar el correcto funcionamiento de la persistencia y las transacciones, destacando la prueba de transferencias por garantizar la consistencia de los datos.

Aunque el sistema aún no está completo, los avances reflejan su gran potencial y capacidad de crecimiento en futuras versiones.

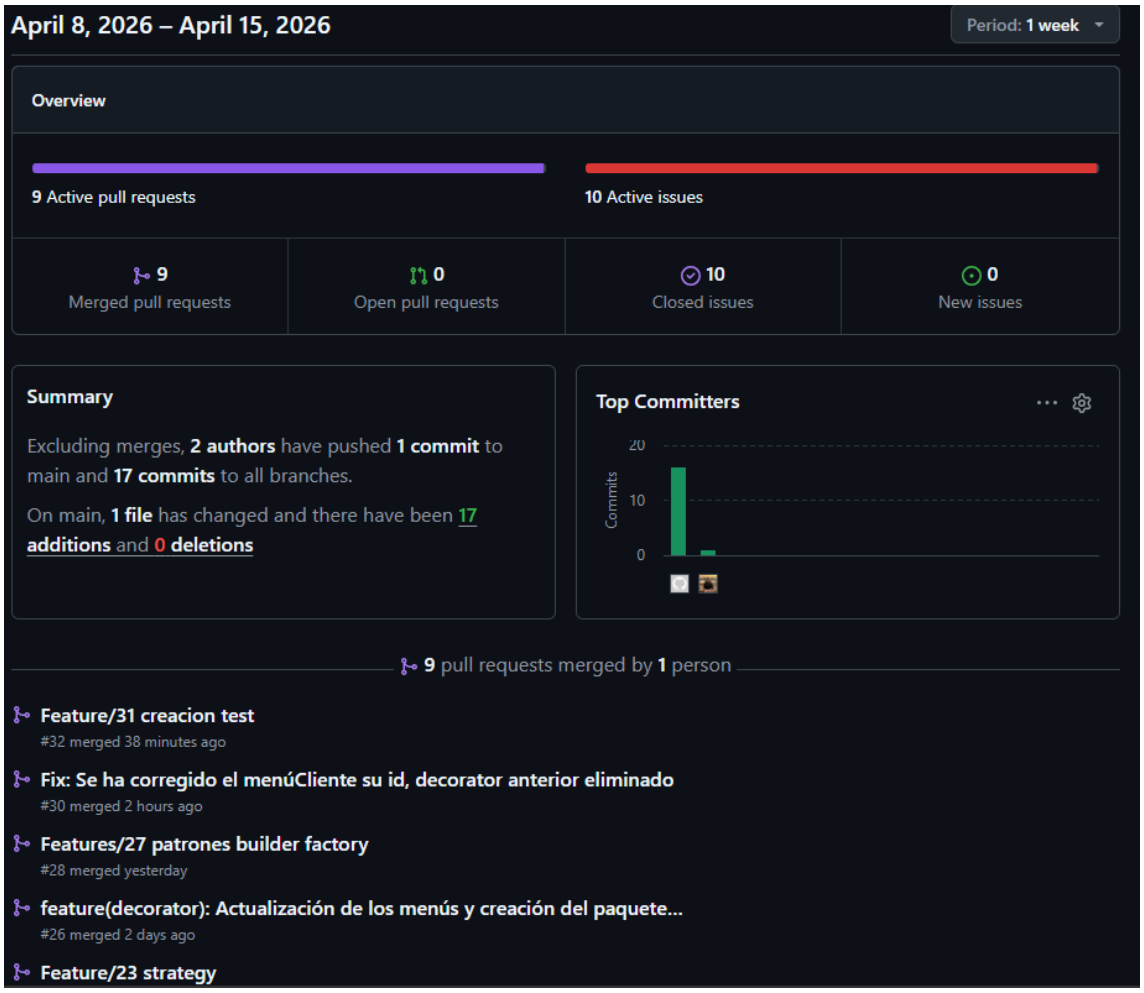

Figura 10.1: Actividad de GitHub para CP2

Bibliografía

[Docs, 2024] Docs, M. (2024). Layered architecture.

[Gamma et al., 1994] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Referencia general de patrones GoF.

[Guru, 2024a] Guru, R. (2024a). Builder design pattern.

[Guru, 2024b] Guru, R. (2024b). Factory method design pattern.

[Guru, 2024c] Guru, R. (2024c). Singleton design pattern.

[Guru, 2024d] Guru, R. (2024d). Strategy design pattern.

[Thakare and Kiwelekar, 2021] Thakare, S. and Kiwelekar, A. W. (2021). Redefining measures of layered architecture. *arXiv preprint*. arXiv:2106.03079.

[Torres et al., 2017] Torres, E., Delgado Dapena, M. D., and Lodos Vigil, J. (2017). Unit tests of software in a university environment. *Polibits*.

[Zhao et al., 2024] Zhao, Y., Xiao, L., Wei, C., Kazman, R., and Yang, Y. (2024). A systematic mapping study on architectural approaches to software performance analysis. *arXiv preprint*. arXiv:2410.17372.